Communications Numériques 1

Master ELISE - 2025-2026

TDAO 1 - Compte-rendu

Synthèse et analyse de filtres numériques RIF et RII

Dalina CHABANE

Master ELISE

Table of contents

Introduction **2**

1 Méthode de synthèse de filtres à RIF **2**
1.1 Filtre à phase linéaire 2
1.2 Réponse impulsionnelle du filtre passe-bas analogique idéal 3
1.3 Séquence échantillonnée $h(n)$ 3
1.4 Réponse fréquentielle de $h(n)$ 4
1.5 Problèmes de bornitude et de causalité 4
1.6 Troncature par fenêtre rectangulaire 4
1.7 Effet de N sur la réponse fréquentielle 5
1.8 Fenêtres d'apodisation 7
1.9 Résolution du problème de causalité 9
1.10 Application à un signal polychromatique transitoire 10

2 Filtre RIF à phase linéaire **12**
2.1 Démonstration de la phase linéaire pour N impair 12
2.2 Gain unitaire et réponse impulsionnelle calibrée 14

3 Partie C : Implémentation sous Matlab et analyse des résultats **15**
3.1 Script de synthèse et d'analyse 15
3.2 Analyse temporelle : La réponse impulsionnelle 15
3.3 Analyse fréquentielle : Module et Temps de propagation 16

4 Partie D : Synthèse de filtres à RIF sous Matlab **17**
4.1 Implémentation et choix des paramètres 17
4.2 Analyse de la réponse fréquentielle 17
4.3 Réponse impulsionnelle et temps de propagation de groupe 18
4.4 Validation pratique par filtrage audio 18

5 Partie E : Synthèse de filtres à RII sous Matlab **18**
5.1 Choix des paramètres et implémentation 18
5.2 Analyse de la réponse temporelle et fréquentielle 20
5.3 Étude de la stabilité : Plan complexe 22
5.4 Validation pratique par filtrage audio 22

Conclusion **23**

Annexes — Codes Matlab **25**

Introduction

Les systèmes de communication numérique reposent en grande partie sur la capacité à traiter, filtrer et analyser des signaux discrets avec précision et efficacité. Au cœur de cette chaîne de traitement se trouvent les **filtres numériques**, dont la conception constitue un enjeu fondamental tant sur le plan théorique que pratique.

Ce travail dirigé a pour objectif de maîtriser les différentes méthodes de synthèse de filtres numériques, en partant des fondements mathématiques jusqu'à leur implémentation et validation sous Matlab. La démarche suivie s'articule en cinq étapes complémentaires.

Dans un premier temps, nous étudions la **méthode de la fenêtre** pour la synthèse de filtres à Réponse Impulsionnelle Finie (RIF), en analysant l'impact du nombre de coefficients N et du choix de la fenêtre d'apodisation sur les performances fréquentielles du filtre obtenu. Nous montrons en particulier comment la troncature de la réponse impulsionnelle idéale introduit le phénomène de Gibbs, et comment les fenêtres pondérées permettent de le réduire au prix d'un élargissement de la bande de transition.

Dans un second temps, nous démontrons rigoureusement pourquoi une **réponse impulsionnelle symétrique** garantit une phase linéaire, propriété essentielle pour éviter toute déformation temporelle du signal filtré. Nous établissons notamment l'expression du temps de propagation de groupe constant associé.

Nous appliquons ensuite ces résultats théoriques à un **exemple concret de synthèse d'un filtre passe-bande** à $N = 9$ coefficients, dont nous calculons analytiquement les coefficients et analysons les performances fréquentielles effectives.

Les deux dernières parties sont consacrées à la synthèse assistée par Matlab, d'abord d'un **filtre RIF optimal** via l'algorithme de Parks-McClellan, puis d'un **filtre RII de Chebyshev de type 2**, dont nous comparons les performances avec le filtre RIF. Une validation pratique sur un signal audio réel clôt chaque partie.

L'ensemble des codes Matlab développés sont regroupés en annexe, et référencés dans le texte au fur et à mesure de leur utilisation.

1 Méthode de synthèse de filtres à RIF

L'avantage des filtres à réponse impulsionnelle finie (RIF) est qu'ils sont **toujours stables**. De plus, on peut construire des filtres à **phase linéaire**. La méthode la plus courante pour synthétiser un filtre RIF est la *méthode de la fenêtre* : partant du principe de réalisation d'un filtrage par convolution, on déduit la réponse impulsionnelle par le calcul de la transformée de Fourier inverse de la réponse fréquentielle désirée.

1.1 Filtre à phase linéaire

Un filtre est dit à **phase linéaire** lorsque son temps de propagation de groupe est *constant*, c'est-à-dire indépendant de la fréquence :

$$\tau_g(f) = -\frac{1}{2\pi}\frac{\mathrm{d}}{\mathrm{d}f}\varphi(f) = \text{constante}$$

Cela implique que la phase $\varphi(f)$ de la réponse fréquentielle varie **linéairement** avec f. En pratique, cette propriété garantit que toutes les composantes fréquentielles du signal subissent le *même retard*, évitant ainsi toute déformation temporelle du signal filtré.

1.2 Réponse impulsionnelle du filtre passe-bas analogique idéal

On considère le filtre passe-bas analogique idéal de fréquence de coupure F_c, dont la réponse fréquentielle est :

$$H_a(f) = \begin{cases} 1 & \text{si } |f| \leq F_c \\ 0 & \text{sinon} \end{cases}$$

Sa réponse impulsionnelle $h_a(t)$ est obtenue par transformée de Fourier inverse :

$$\begin{aligned} h_a(t) &= \int_{-\infty}^{+\infty} H_a(\nu)\, e^{j2\pi\nu t}\, \mathrm{d}\nu = \int_{-F_c}^{+F_c} e^{j2\pi\nu t}\, \mathrm{d}\nu \\ &= \left[\frac{e^{j2\pi\nu t}}{j2\pi t}\right]_{-F_c}^{+F_c} = \frac{e^{j2\pi F_c t} - e^{-j2\pi F_c t}}{j2\pi t} \\ &= \frac{2j\sin(2\pi F_c t)}{j2\pi t} \\ &= \frac{2\pi F_c \cdot \sin(2\pi F_c t)}{2\pi F_c \cdot \pi t} \end{aligned}$$

$$\boxed{h_a(t) = \frac{\sin(2\pi F_c t)}{\pi t} = 2F_c \operatorname{sinc}(2\pi F_c t)}$$

Cette fonction est **symétrique** (paire) et s'étend sur $t \in \,]-\infty, +\infty[$: elle n'est donc ni causale ni à support borné.

1.3 Séquence échantillonnée $h(n)$

Pour synthétiser un filtre numérique, on échantillonne $h_a(t)$ à la fréquence F_e. L'opération d'échantillonnage revient à multiplier $h_a(t)$ par un peigne de Dirac de période $T_e = 1/F_e$:

$$h_a(t) \cdot \Delta_{T_e}(t) = h_a(t) \cdot \sum_{n=-\infty}^{+\infty} \delta(t - nT_e) = \sum_{n=-\infty}^{+\infty} h_a(nT_e)\, \delta(t - nT_e)$$

Problème de normalisation. On vérifie que :

$$\int_{-\infty}^{+\infty} h_a(t)\, \mathrm{d}t = H_a(0) = 1 \qquad \text{mais} \qquad \int_{-\infty}^{+\infty} h_a(t) \cdot \Delta_{T_e}(t)\, \mathrm{d}t = F_e \cdot H_a(0) = F_e$$

Il y a un facteur $F_e = 1/T_e$ d'écart. Pour conserver un gain unitaire en $f = 0$, la séquence doit être pondérée par T_e :

$$h(n) = T_e \cdot h_a(nT_e) = T_e \cdot \frac{\sin(2\pi F_c\, nT_e)}{\pi\, nT_e} = \frac{\sin(2\pi F_c\, nT_e)}{\pi\, n}$$

$$\boxed{h(n) = 2\frac{F_c}{F_e} \operatorname{sinc}\left(2\pi n \frac{F_c}{F_e}\right), \quad n \in Z}$$

avec $h(0) = 2F_c/F_e$ obtenu par la limite $\lim_{n\to 0} \operatorname{sinc}(x) = 1$.

1.4 Réponse fréquentielle de $h(n)$

L'échantillonnage dans le domaine temporel entraîne une **périodisation du spectre** dans le domaine fréquentiel. En utilisant $\mathcal{TF}[\Delta_{T_e}(t)] = F_e \cdot \Delta_{F_e}(f)$ et la propriété de convolution :

$$H(f) = T_e \cdot H_a(f) * \mathcal{TF}[\Delta_{T_e}(t)] = T_e \cdot H_a(f) * F_e\, \Delta_{F_e}(f)$$

$$\boxed{H(f) = \sum_{k=-\infty}^{+\infty} H_a(f - kF_e)}$$

Le spectre idéal $H_a(f)$ se répète périodiquement tous les F_e Hz.

1.5 Problèmes de bornitude et de causalité

La séquence $h(n) = 2(F_c/F_e)\,\mathrm{sinc}(2\pi n F_c/F_e)$ présente deux problèmes fondamentaux qui la rendent non implémentable :

1. **Bornitude (support infini).** $h(n) \neq 0$ pour tout $n \in Z$: la séquence ne s'annule pas définitivement, son amplitude décroît seulement en $1/|n|$. Il faudrait stocker une infinité de coefficients, ce qui est physiquement impossible.
2. **Causalité.** $h(n) \neq 0$ pour $n < 0$, ce qui signifie que la sortie dépendrait d'échantillons *futurs* de l'entrée ($Z \neq N$). Un filtre implémentable en temps réel doit vérifier $h(n) = 0$ pour tout $n < 0$.

1.6 Troncature par fenêtre rectangulaire

Pour résoudre le problème de bornitude, on multiplie $h(n)$ par une fenêtre rectangulaire $w_{\text{rect}}(n)$ de longueur N :

$$w_{\text{rect}}(n) = \begin{cases} 1 & \text{si } -\dfrac{N-1}{2} \leq n \leq \dfrac{N-1}{2} \quad (N \text{ impair}) \\ 0 & \text{sinon} \end{cases}$$

La séquence tronquée est :

$$\bar{h}(n) = h(n) \cdot w_{\text{rect}}(n)$$

Par la propriété produit/convolution de la transformée de Fourier :

$$\bar{H}(f) = W_{\text{rect}}(f) * H(f)$$

La TF de la fenêtre rectangulaire est :

$$W_{\text{rect}}(f) = \sum_{n=-(N-1)/2}^{(N-1)/2} e^{-j2\pi f n T_e}$$

En posant $u = e^{-j2\pi f T_e}$, on reconnaît une **somme géométrique** de N termes :

$$W_{\text{rect}}(f) = u^{-(N-1)/2} \cdot \sum_{k=0}^{N-1} u^k = u^{-(N-1)/2} \cdot \frac{u^N - 1}{u - 1}$$

On factorise numérateur et dénominateur par $u^{N/2}$ et $u^{1/2}$ respectivement :

$$W_{\text{rect}}(f) = u^{-(N-1)/2} \cdot u^{(N-1)/2} \cdot \frac{u^{N/2} - u^{-N/2}}{u^{1/2} - u^{-1/2}} = \frac{u^{N/2} - u^{-N/2}}{u^{1/2} - u^{-1/2}}$$

En utilisant la définition du sinus $e^{jx} - e^{-jx} = 2j\sin(x)$ et en substituant $u = e^{-j2\pi fT_e}$:

$$\text{Numérateur : } u^{N/2} - u^{-N/2} = e^{-j\pi NfT_e} - e^{+j\pi NfT_e} = -2j\sin(\pi NfT_e)$$

$$\text{Dénominateur : } u^{1/2} - u^{-1/2} = e^{-j\pi fT_e} - e^{+j\pi fT_e} = -2j\sin(\pi fT_e)$$

Les facteurs $-2j$ se simplifient :

$$\boxed{W_{\text{rect}}(f) = \frac{\sin(\pi NfT_e)}{\sin(\pi fT_e)}}$$

Pour $fT_e \ll 1$, on utilise l'approximation $\sin(\pi fT_e) \approx \pi fT_e$, ce qui donne :

$$W_{\text{rect}}(f) \approx \frac{\sin(\pi NfT_e)}{\pi fT_e} = N \cdot \frac{\sin\left(\frac{\pi Nf}{F_e}\right)}{\frac{\pi Nf}{F_e}}$$

$$\boxed{W_{\text{rect}}(f) \approx N \cdot \text{sinc}\left(\frac{\pi Nf}{F_e}\right)}$$

L'expression complète de $\bar{H}(f)$ est donc :

$$\boxed{\bar{H}(f) = N \cdot \text{sinc}\left(\frac{\pi Nf}{F_e}\right) * \sum_{k=-\infty}^{+\infty} H_a(f - kF_e)}$$

1.7 Effet de N sur la réponse fréquentielle

Dans cette partie pratique, nous avons simulé la réponse impulsionnelle d'un filtre passe-bas idéal (sinus cardinal) pour observer l'impact de la troncature. Nous avons testé trois tailles de fenêtres rectangulaires différentes : $N \in \{16, 51, 101\}$.

Le code Matlab correspondant est fourni en Annexe 5.4 (page 25).

Analyse de la réponse temporelle

Sur la Figure 1, la simulation valide notre calcul théorique : la réponse impulsionnelle d'un filtre passe-bas idéal prend bien la forme d'un sinus cardinal.

On remarque que plus le nombre d'échantillons N est grand, plus on conserve d'informations sur les extrémités du signal. Cependant, les indices de temps sont négatifs (le signal est centré sur $n = 0$). Le filtre simulé nécessite donc de connaître des échantillons futurs pour calculer le présent : il est non-causal et irréalisable physiquement tel quel.

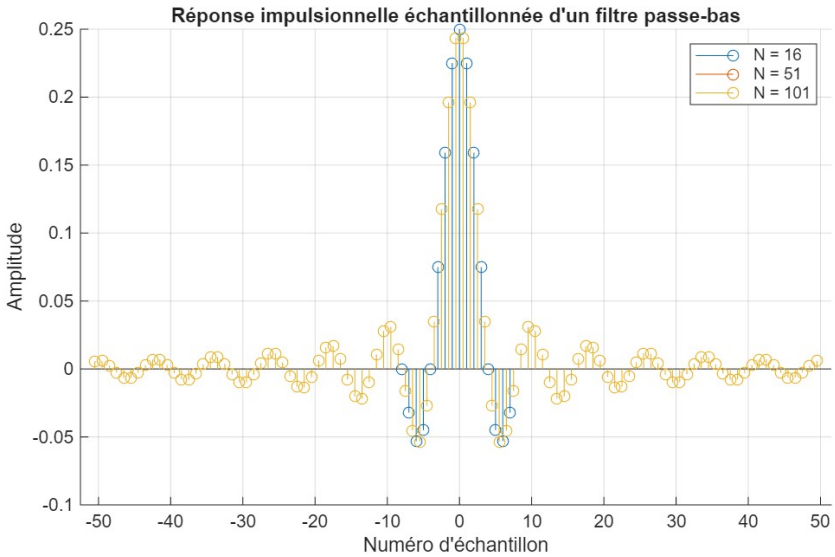

FIGURE 1 – Troncature du sinus cardinal pour $N = 16$, 51 et 101

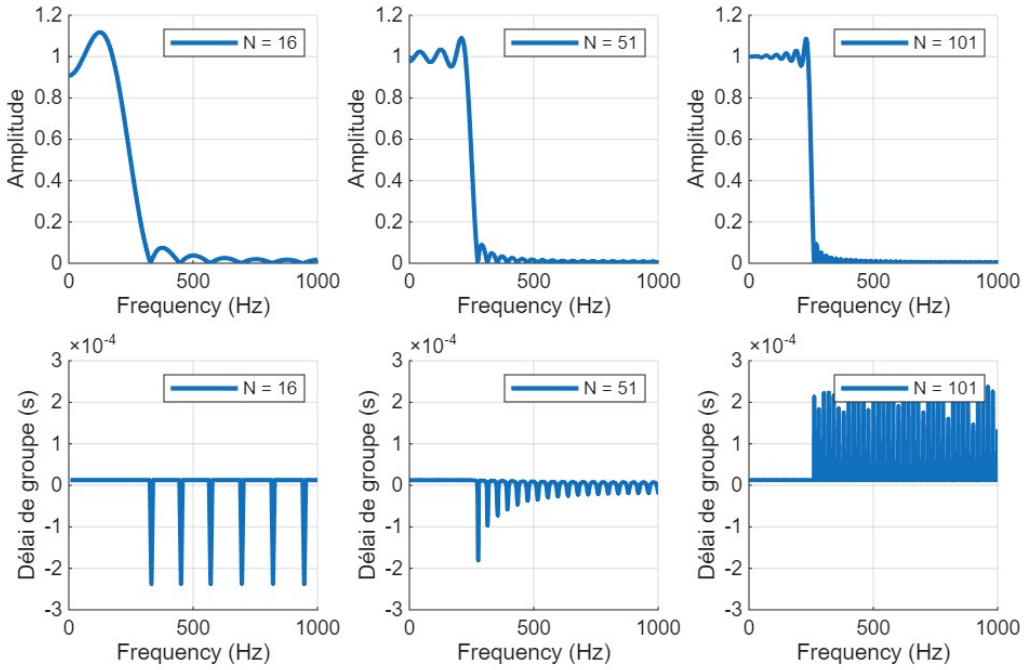

FIGURE 2 – Module (haut) et temps de propagation de groupe (bas) selon N

Analyse fréquentielle et phénomène de Gibbs

La Figure 2 met en évidence l'effet de la troncature brutale (fenêtre rectangulaire) dans le domaine fréquentiel. Les résultats montrent un comportement conforme à la théorie du phénomène de Gibbs :

— **Sur la zone de transition :** Plus la taille de la fenêtre N augmente, plus la pente de coupure devient raide. Le filtre se rapproche du gabarit idéal.

— **Sur les ondulations (Ripples) :** Lorsque N augmente, le nombre d'ondulations augmente proportionnellement. En revanche, l'amplitude maximale de ces ondula-

tions reste strictement la même (elle ne s'écrase pas). C'est la limite principale de la fenêtre rectangulaire.

Enfin, concernant les graphiques du bas, le temps de propagation de groupe mesuré est de 0 (aux erreurs numériques près). Le filtre n'induit aucun retard car son centre de symétrie temporel est situé en 0. Il s'agit bien d'un filtre à phase nulle.

1.8 Fenêtres d'apodisation

Pour réduire les artefacts de troncature observés à la partie précédente, on remplace la fenêtre rectangulaire par une fenêtre pondérée qui s'annule progressivement aux bords. Le Tableau 1 récapitule les principales fenêtres utilisées ainsi que l'expression de leur séquence $w(n)$.

Sous Matlab, on peut tracer les réponses impulsionnelles ainsi que les fonctions de transfert des fenêtres d'apodisation présentées précédemment. Les résultats se trouvent sur les Figures 3 et 4.

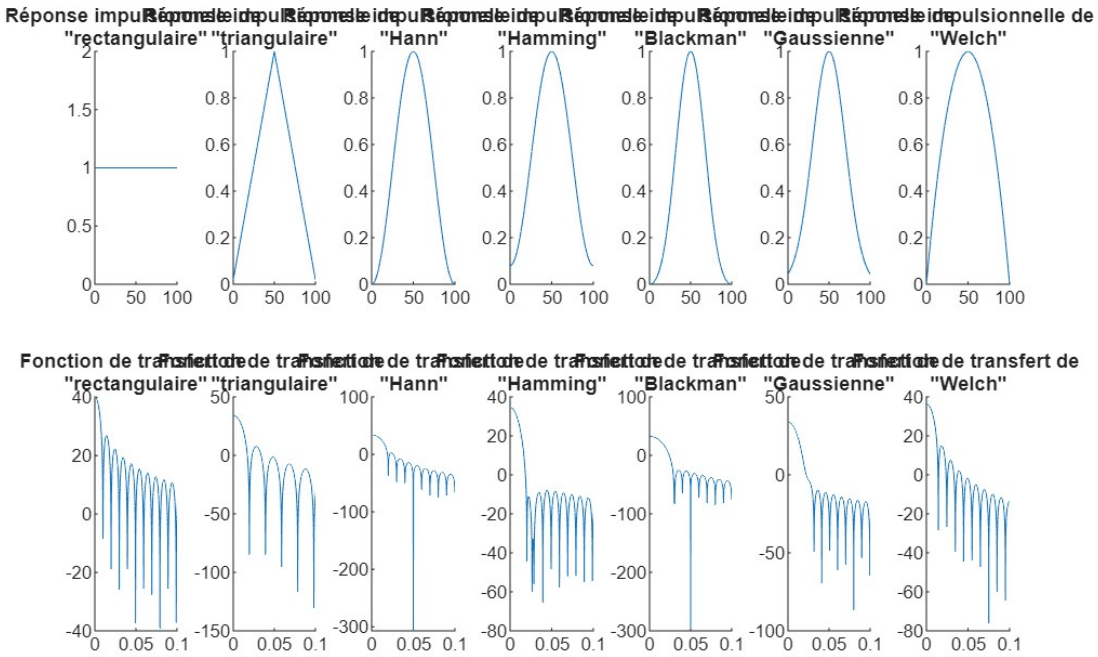

FIGURE 3 – Réponse impulsionnelle et fonction de transfert des 7 premières fenêtres d'apodisation

On implémente ensuite des fonctions Matlab permettant de déterminer automatiquement la largeur du premier lobe, l'atténuation des lobes secondaires par rapport au lobe principal, et la pente de la zone de Gibbs. Cette dernière correspond à la pente entre les pics de gain de la fonction de transfert (sans le premier lobe). On voit sur les graphiques des Figures 3 et 4 que le gain du lobe principal n'a pas de pic ; le *worst-case loss* n'a donc pas d'intérêt à être calculé ici, contrairement au Tableau 2 qui présente les valeurs théoriques de référence à titre indicatif.

La largeur du premier lobe correspond à la recherche de l'indice du premier minimum, tandis que l'atténuation des lobes secondaires par rapport au lobe principal correspond à la recherche des deux premiers maximaux locaux. La pente de la zone de Gibbs est

Table 1 – Expressions mathématiques exhaustives des séquences $w(n)$ des fenêtres d'apodisation

Fenêtre	**Expression** $w(n)$
Rectangulaire	$w(n) = 1$
Triangulaire	$w(n) = \frac{2}{N-1} \cdot \left(\frac{N-1}{2} - \left\lvert n - \frac{N-1}{2} \right\rvert\right)$
Hann	$w(n) = \frac{1}{2}\left(1 - \cos\left(\frac{2\pi n}{N-1}\right)\right)$
Hamming	$w(n) = 0{,}54 - 0{,}46 \cdot \cos\left(\frac{2\pi n}{N-1}\right)$
Blackman	$w(n) = a_0 - a_1 \cdot \cos\left(\frac{2\pi n}{N-1}\right) + a_2 \cdot \cos\left(\frac{4\pi n}{N-1}\right)$ (Avec $a_0 = 0{,}42$, $a_1 = 0{,}5$, $a_2 = 0{,}08$ canoniquement)
Gauss	$w(n) = e^{-\frac{1}{2}\left(\frac{n-\frac{N-1}{2}}{\sigma\frac{N-1}{2}}\right)^2}$
Welch	$w(n) = 1 - \left(\frac{n-\frac{N}{2}}{\frac{N}{2}}\right)^2 \quad \text{pour } 0 \leq n \leq N$
Puissance de sinus	$w(n) = \sin\left(\frac{\pi n}{N}\right) \quad \text{pour } 0 \leq n \leq N$
Flat top	$w(n) = a_0 - a_1 \cos\left(\frac{2\pi n}{N}\right) + a_2 \cos\left(\frac{4\pi n}{N}\right) - a_3 \cos\left(\frac{6\pi n}{N}\right) + a_4 \cos\left(\frac{8\pi n}{N}\right)$ Avec : $a_0 = 0{,}21557$; $a_1 = 0{,}41663$; $a_2 = 0{,}27726$; $a_3 = 0{,}08357$; $a_4 = 0{,}00694$
Tukey	$w(n) = \begin{cases} 1 & \text{si } 0 \leq \lvert n \rvert \leq \frac{\alpha N}{2} \\ \frac{1}{2}\left[1 + \cos\left(\pi \frac{\lvert n \rvert - \alpha\frac{N}{2}}{(1-\alpha)\frac{N}{2}}\right)\right] & \text{si } \frac{\alpha N}{2} \leq \lvert n \rvert \leq \frac{N}{2} \end{cases}$
Lanczos	$w(n) = \operatorname{sinc}\left(\frac{2n}{N-1} - 1\right)$
Kaiser	$w(n) = \frac{I_0\left(\pi\alpha\sqrt{1-\left(\frac{2n}{N-1}-1\right)^2}\right)}{I_0(\pi\alpha)}$
Blackman-Harris	$w(n) = a_0 - a_1 \cos\left(\frac{2\pi n}{N-1}\right) + a_2 \cos\left(\frac{4\pi n}{N-1}\right) - a_3 \cos\left(\frac{6\pi n}{N-1}\right)$ Avec : $a_0 = 0{,}35875$; $a_1 = 0{,}48829$; $a_2 = 0{,}14128$; $a_3 = 0{,}01168$
Blackman-Nuttall	$w(n) = a_0 - a_1 \cos\left(\frac{2\pi n}{N-1}\right) + a_2 \cos\left(\frac{4\pi n}{N-1}\right) - a_3 \cos\left(\frac{6\pi n}{N-1}\right)$ Avec : $a_0 = 0{,}355768$; $a_1 = 0{,}487396$; $a_2 = 0{,}144232$; $a_3 = 0{,}012604$

plus complexe à déterminer : on utilise la fonction `findpeaks` pour récupérer directement les pics ainsi que leurs indices. Il est parfois nécessaire d'ajuster quels pics sont utilisés pour le calcul, car par exemple la fenêtre de Hamming présente un troisième pic de faible amplitude par rapport aux autres.

Le code Matlab correspondant est fourni en **Annexe 5.4 (page 26)**. Les résultats

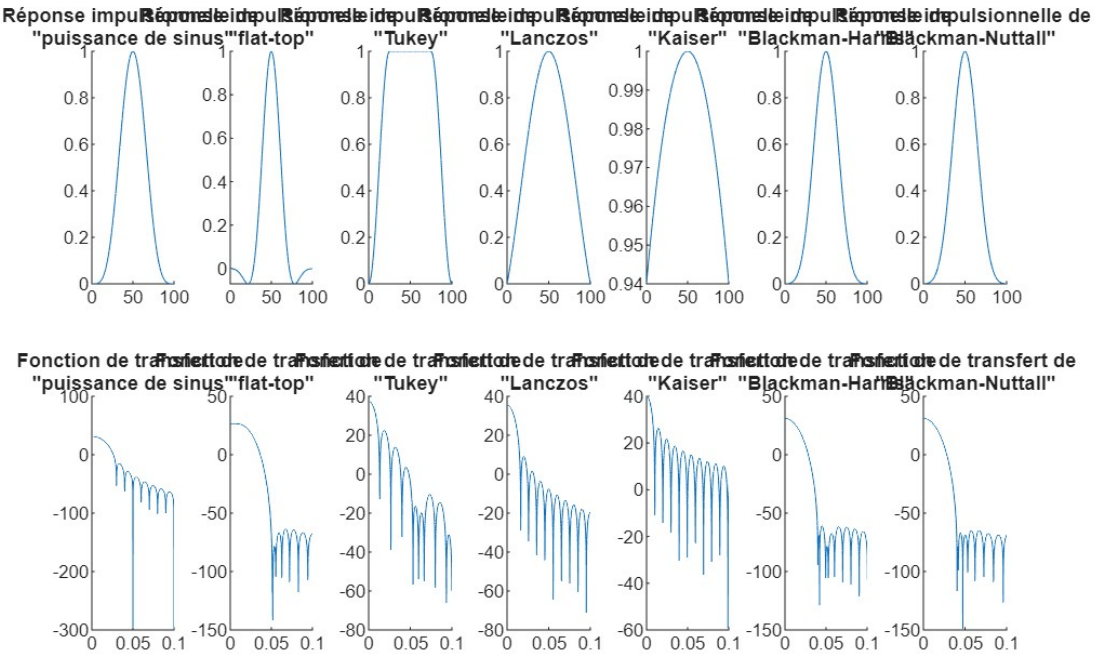

FIGURE 4 – Réponse impulsionnelle et fonction de transfert des 7 dernières fenêtres d'apodisation

des performances des fenêtres se trouvent dans le Tableau 2.

Les valeurs obtenues ne correspondent pas toujours exactement aux valeurs théoriques en raison du nombre de points choisi ($N = 101$) et des approximations inhérentes à l'algorithme d'extraction des pics. Cependant, elles restent globalement cohérentes avec les références et permettent une comparaison pertinente entre les différentes fenêtres.

On peut noter par exemple que la fenêtre **Flat-top** est la plus large de toutes, tandis que les fenêtres rectangulaire et Kaiser sont les plus étroites. Les fenêtres Hann et Blackman présentent une pente de la zone de Gibbs bien plus importante que la fenêtre rectangulaire. Concernant l'atténuation des lobes secondaires, les trois fenêtres qui se distinguent nettement sont la **Flat-top**, la **Blackman-Harris** et la **Blackman-Nuttall**.

: Les fenêtres d'apodisation permettent de modifier la réponse du filtre afin d'obtenir les performances souhaitées. De nombreuses fenêtres existent, chacune avec ses avantages et ses défauts : certaines permettent d'obtenir un lobe principal très étroit pour une meilleure sélectivité en fréquence, d'autres offrent une forte pente dans la zone de Gibbs, ou encore une importante atténuation des lobes secondaires pour un meilleur rejet hors bande. Le compromis fondamental reste inévitable : plus la fenêtre atténue les lobes secondaires, plus son lobe central est large, ce qui élargit la bande de transition.clau

1.9 Résolution du problème de causalité

Pour rendre $\bar{h}(n)$ causale, on lui applique un décalage de $(N-1)/2$ échantillons vers la droite :

$$\hat{h}(n) = \bar{h}\left(n - \frac{N-1}{2}\right), \quad n \in \{0, 1, \ldots, N-1\}$$

Table 2 – Récapitulatif des performances des différentes fenêtres d'apodisation

Fenêtre	Largeur du lobe central (Bins/N)	Pente de la zone de Gibbs (dB/oct)	Atténuation des lobes sec. (dB)	Worst-Cas Loss (dB
Rectangulaire	2	-6	-13	3,92
Triangulaire	4	-12	-27	1,82
Welch	$\approx 2{,}6$	-6	-21	2,12
Puissance de sinus	3	-12	-23	2,34
Hann	4	-18	-32	1,42
Hamming	4	-6	-43	1,78
Blackman	6	-18	-58	1,10
Lanczos	$\approx 3{,}2$	-12	-26	1,80
Tukey ($\alpha = 0{,}5$)	$\approx 2{,}75$	-18	-15	2,20
Gauss ($\sigma = 0{,}4$)	$\approx 3{,}4$	-6	-42	$\approx 1{,}60$
Kaiser ($\beta = 3$)	$\approx 3{,}4$	-6	-24	$\approx 1{,}50$
Flat-top	≈ 5	-6	-93	$< 0{,}01$
Blackman-Harris	8	-6	-92	0,83
Blackman-Nuttall	8	-18	-98	0,83

Dans le domaine fréquentiel, un décalage temporel de d échantillons introduit un déphasage linéaire (propriété de retard de la TF) :

$$\hat{H}(f) = \bar{H}(f) \cdot e^{-j\pi(N-1)T_e f}$$

Ce qui change et ce qui ne change pas :

- Le **module** reste inchangé : $|\hat{H}(f)| = |\bar{H}(f)|$. Le filtre filtre identiquement.
- La **phase** devient linéaire : $\angle\hat{H}(f) = \angle\bar{H}(f) - \pi(N-1)T_e\, f$.
- Le **temps de propagation de groupe** est constant :

$$\boxed{\tau_g(f) = -\frac{1}{2\pi}\frac{\mathrm{d}}{\mathrm{d}f}\angle\hat{H}(f) = \frac{N-1}{2}\,T_e}$$

Toutes les fréquences subissent le même retard de $(N-1)/2$ échantillons, ce qui garantit l'absence de déformation du signal filtré.

1.10 Application à un signal polychromatique transitoire

Pour clore cette partie sur le fenêtrage, nous avons appliqué quatre fenêtres d'apodisation différentes (Rectangulaire, Flat-top, Tukey et Blackman-Harris) sur un signal polychromatique.

L'objectif de cette manipulation est de mettre en évidence le phénomène de **fuite spectrale** (spectral leakage) et la manière dont chaque fenêtre permet de le corriger.

Nous avons volontairement choisi un signal contenant des fréquences qui ne sont pas des multiples entiers de notre durée d'observation (1 s), soit $f_1 = 12{,}5\,\text{Hz}$ et $f_3 = 110{,}1\,\text{Hz}$, en plus d'une fréquence "juste" $f_2 = 50\,\text{Hz}$.

Le code Matlab correspondant est fourni en Annexe 5.4 (page 29).

Analyse du signal original

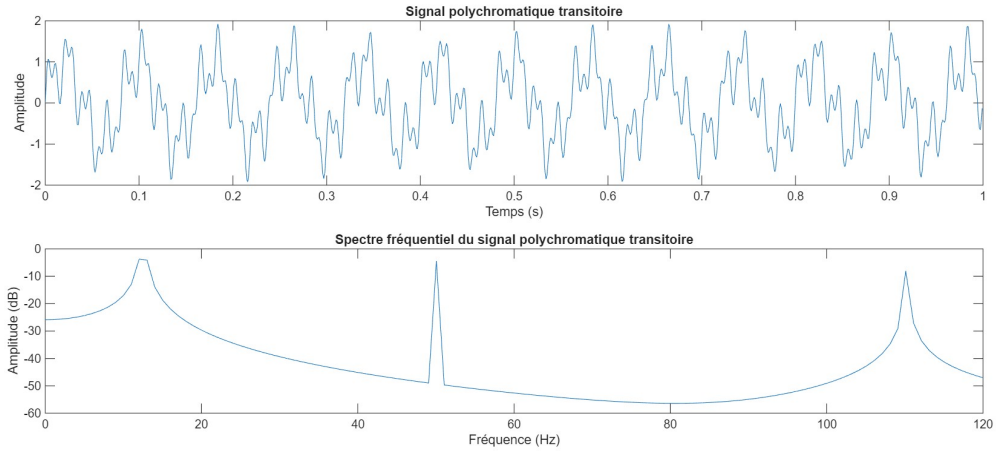

FIGURE 5 – Signal polychromatique brut et son spectre (Fenêtre rectangulaire par défaut)

La Figure 5 montre le signal généré. Puisque les fréquences 12,5 Hz et 110,1 Hz ne complètent pas un nombre entier de périodes sur notre fenêtre de 1 s, le signal subit une coupure brutale à ses extrémités.

Dans le domaine fréquentiel, cela se traduit par une forte **fuite spectrale** : l'énergie des pics s'étale sur les fréquences voisines, créant des "jupes" très larges à la base des composantes fréquentielles.

Comparaison des fenêtres d'apodisation

L'application des différentes fenêtres (Figure 6) illustre parfaitement leurs cas d'usage respectifs :

— **Fenêtre Rectangulaire :** Elle offre la résolution en fréquence la plus fine, mais la fuite spectrale est maximale. Il devient difficile de lire l'amplitude exacte des composantes f_1 et f_3.

— **Fenêtre Flat-top :** Temporellement, elle écrase fortement le signal. Fréquentiellement, ses pics sont très larges (mauvaise résolution). En revanche, comme son nom l'indique, le sommet de son lobe est plat. C'est la fenêtre idéale pour mesurer l'**amplitude exacte** d'une raie fréquentielle, même si celle-ci ne tombe pas parfaitement sur un point de calcul de la FFT.

— **Fenêtre de Tukey :** C'est un compromis intéressant. Elle ne modifie que les bords du signal temporel. Le pic central est fin et la fuite spectrale est légèrement réduite par rapport à la rectangulaire.

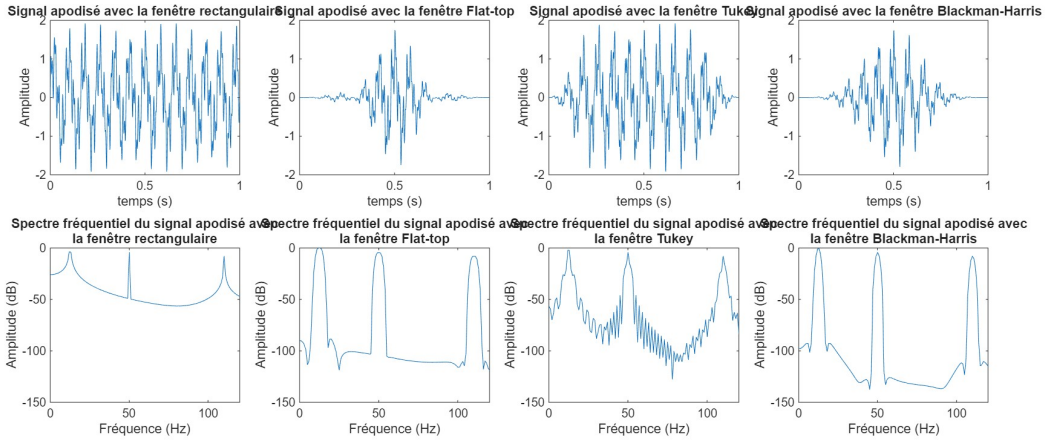

FIGURE 6 – Comparaison des signaux temporels et des spectres apodisés

— **Fenêtre de Blackman-Harris :** Grâce à l'atténuation colossale de ses lobes secondaires (vue à la Question 8), elle écrase totalement la fuite spectrale. Le plancher de bruit spectral s'effondre en dessous des $-100\,\text{dB}$. C'est la fenêtre à privilégier pour distinguer un signal de très faible amplitude situé à proximité d'un signal très fort.

Conclusion

En conclusion, la simulation montre qu'il n'existe pas de fenêtre "parfaite". Le choix de la fenêtre d'apodisation dépend toujours du but recherché : si l'on souhaite séparer deux fréquences très proches, on privilégiera une fenêtre **Rectangulaire** au détriment de la précision d'amplitude. Si l'on cherche l'amplitude exacte des harmoniques, on utilisera une **Flat-top**. Enfin, si l'on souhaite observer des signaux de très faible dynamique noyés dans le bruit, une **Blackman-Harris** sera le choix le plus pertinent.

2 Filtre RIF à phase linéaire

2.1 Démonstration de la phase linéaire pour N impair

Condition de symétrie

On suppose que $\hat{h}(n)$ est **causale** (définie sur $n \in \{0, 1, \ldots, N-1\}$) et **symétrique** par rapport à son coefficient central :

$$\hat{h}(n) = \hat{h}(N-1-n) \qquad \forall\, n \in \{0, 1, \ldots, N-1\}$$

Pour $N = 9$ par exemple, le coefficient central est en $n = 4$ et la symétrie s'exprime : $\hat{h}(0) = \hat{h}(8)$, $\hat{h}(1) = \hat{h}(7)$, $\hat{h}(2) = \hat{h}(6)$, $\hat{h}(3) = \hat{h}(5)$.

Expression 1 — définition directe

La réponse fréquentielle s'obtient en évaluant $H(z)$ sur le cercle unité $z = e^{j2\pi f T_e}$:

$$\hat{H}(f) = \sum_{n=0}^{N-1} \hat{h}(n) \cdot e^{-j2\pi f T_e n} \tag{1}$$

Expression 2 — changement de variable

On effectue le changement de variable $m = N - 1 - n$ dans la somme (1). Quand n parcourt $\{0, \ldots, N-1\}$, m parcourt également $\{0, \ldots, N-1\}$ dans l'ordre inverse, et $n = N - 1 - m$. On obtient :

$$\hat{H}(f) = \sum_{m=0}^{N-1} \hat{h}(N-1-m) \cdot e^{-j2\pi f T_e (N-1-m)} \tag{2}$$

On applique la condition de symétrie $\hat{h}(N-1-m) = \hat{h}(m)$:

$$\hat{H}(f) = \sum_{m=0}^{N-1} \hat{h}(m) \cdot e^{-j2\pi f T_e (N-1)} \cdot e^{+j2\pi f T_e m} = e^{-j2\pi f T_e (N-1)} \cdot \sum_{n=0}^{N-1} \hat{h}(n) \cdot e^{+j2\pi f T_e n} \tag{2'}$$

Puisque $\hat{h}(n)$ est à coefficients **réels**, la somme dans $(2')$ est le conjugué complexe de celle dans (1) :

$$\sum_{n=0}^{N-1} \hat{h}(n) \cdot e^{+j2\pi f T_e n} = \hat{H}^*(f) \qquad \Longrightarrow \qquad \hat{H}(f) = e^{-j2\pi f T_e (N-1)} \cdot \hat{H}^*(f) \tag{3}$$

Addition de (1) et (1')

On multiplie les deux membres de (1) par $e^{j2\pi f T_e (N-1)}$:

$$\hat{H}(f) \cdot e^{j2\pi f T_e (N-1)} = \sum_{n=0}^{N-1} \hat{h}(n) \cdot e^{-j2\pi f T_e n} \cdot e^{j2\pi f T_e (N-1)} = \sum_{n=0}^{N-1} \hat{h}(n) \cdot e^{j2\pi f T_e (N-1-n)} \tag{1'}$$

On additionne (1) et (1') membre à membre :

$$\hat{H}(f) \cdot \left[1 + e^{j2\pi f T_e (N-1)}\right] = \sum_{n=0}^{N-1} \hat{h}(n) \cdot \left[e^{-j2\pi f T_e n} + e^{+j2\pi f T_e (N-1-n)}\right] \tag{4}$$

On applique la condition de symétrie sur le second terme de la somme (changement de variable $k = N - 1 - n$, puis $\hat{h}(N-1-k) = \hat{h}(k)$) :

$$\sum_{n=0}^{N-1} \hat{h}(n) \cdot e^{+j2\pi f T_e (N-1-n)} = \sum_{n=0}^{N-1} \hat{h}(n) \cdot e^{+j2\pi f T_e n}$$

Le crochet dans (4) devient :

$$e^{-j2\pi f T_e n} + e^{+j2\pi f T_e n} = 2\cos(2\pi f T_e n)$$

On obtient finalement :

$$\boxed{\hat{H}(f) \cdot \left[1 + e^{j2\pi f T_e (N-1)}\right] = \sum_{n=0}^{N-1} \hat{h}(n) \cdot 2\cos(2\pi f T_e n)}$$

Analyse de la phase

Phase du membre gauche. On factorise $\left[1+e^{j2\pi fT_e(N-1)}\right]$ par $e^{j\pi fT_e(N-1)}$:

$$1+e^{j2\pi fT_e(N-1)} = e^{j\pi fT_e(N-1)}\cdot\underbrace{\left(e^{-j\pi fT_e(N-1)}+e^{+j\pi fT_e(N-1)}\right)}_{=2\cos(\pi fT_e(N-1))} = e^{j\pi fT_e(N-1)}\cdot 2\cos(\pi fT_e(N-1))$$

Le cosinus est **réel**, donc sa phase vaut $k\pi$ avec $k \in Z$. La phase du membre gauche est :

$$\angle\left[1+e^{j2\pi fT_e(N-1)}\right] = \pi fT_e(N-1)+k\pi$$

Phase du membre droit. La somme $\sum_{n=0}^{N-1}\hat{h}(n)\cdot 2\cos(2\pi fT_e n)$ est une combinaison linéaire de cosinus à coefficients **réels**. Elle est donc réelle, et sa phase vaut :

$$\angle\left[\sum_{n=0}^{N-1}\hat{h}(n)\cdot 2\cos(2\pi fT_e n)\right] = k'\pi \qquad k' \in Z$$

Phase de $\hat{H}(f)$. De $\hat{H}(f)\cdot X(f) = Y(f)$, on tire $\angle\hat{H}(f) = \angle Y(f) - \angle X(f)$:

$$\boxed{\angle\hat{H}(f) = -\pi fT_e(N-1)+K\pi \qquad K \in Z}$$

La phase est bien **linéaire en** f. Les sauts de $K\pi$ correspondent aux changements de signe du gain réel et ne constituent pas une déformation du signal.

Temps de propagation de groupe

$$\tau_g(f) = -\frac{1}{2\pi}\frac{\mathrm{d}}{\mathrm{d}f}\angle\hat{H}(f) = -\frac{1}{2\pi}\cdot(-\pi T_e(N-1))$$

$$\boxed{\tau_g(f) = \frac{N-1}{2}T_e = \frac{N-1}{2F_e}}$$

Le temps de propagation de groupe est une **constante**, indépendante de f : toutes les fréquences subissent le même retard de $(N-1)/2$ échantillons, ce qui garantit l'absence de déformation du signal.

2.2 Gain unitaire et réponse impulsionnelle calibrée

Vérification du gain en $f = 0$

En $f = 0$, toutes les exponentielles valent 1. D'après (1) :

$$\hat{H}(0) = \sum_{n=0}^{N-1}\hat{h}(n)\cdot e^0 = \sum_{n=0}^{N-1}\hat{h}(n)$$

Le gain à fréquence nulle est égal à la **somme de tous les coefficients**. Il n'y a aucune raison pour que cette somme vaille exactement 1 en général. Si $\hat{H}(0) \neq 1$, le filtre introduit une atténuation ou une amplification non désirée dans la bande passante.

Définition de la réponse impulsionnelle calibrée

Pour garantir un gain unitaire en $f = 0$, on normalise chaque coefficient par la somme totale :

$$\boxed{h_c(n) = \frac{\hat{h}(n)}{\sum_{k=0}^{N-1} \hat{h}(k)}}$$

Vérification :

$$H_c(0) = \sum_{n=0}^{N-1} h_c(n) = \sum_{n=0}^{N-1} \frac{\hat{h}(n)}{\sum_{k=0}^{N-1} \hat{h}(k)} = \frac{\sum_{n=0}^{N-1} \hat{h}(n)}{\sum_{k=0}^{N-1} \hat{h}(k)} = 1$$

La phase linéaire est **conservée** car diviser par une constante réelle ne modifie pas la phase :

$$\angle H_c(f) = \angle \hat{H}(f) = -\pi f T_e (N-1) + K\pi$$

3 Partie C : Implémentation sous Matlab et analyse des résultats

3.1 Script de synthèse et d'analyse

Au cours de cette partie, nous avons complété le script Matlab fourni pour générer la réponse impulsionnelle causale $\hat{h}(n)$ de notre filtre RIF passe-bande ($N = 9$, $f_{c1} = 200\,\text{Hz}$, $f_{c2} = 300\,\text{Hz}$) et extraire ses caractéristiques en fréquence.

Le code Matlab correspondant est fourni en Annexe 5.4 (page 30).

3.2 Analyse temporelle : La réponse impulsionnelle

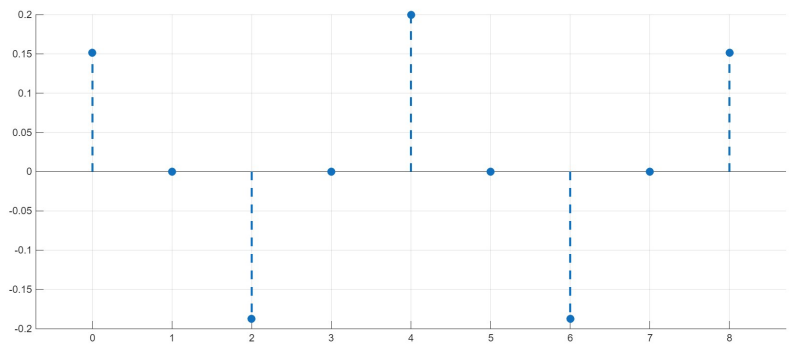

FIGURE 7 – Coefficients de la réponse impulsionnelle causale $\hat{h}(n)$

Sur la Figure 7, on retrouve les 9 coefficients de notre filtre numérique. La simulation valide bien le fait que la séquence a été rendue causale puisqu'elle démarre à $n = 0$.

Ce qu'il est important de remarquer ici, c'est la symétrie parfaite de la séquence par rapport à l'échantillon central ($n = 4$). D'après le cours, c'est cette symétrie qui permet de garantir que le filtre possède une phase linéaire.

3.3 Analyse fréquentielle : Module et Temps de propagation

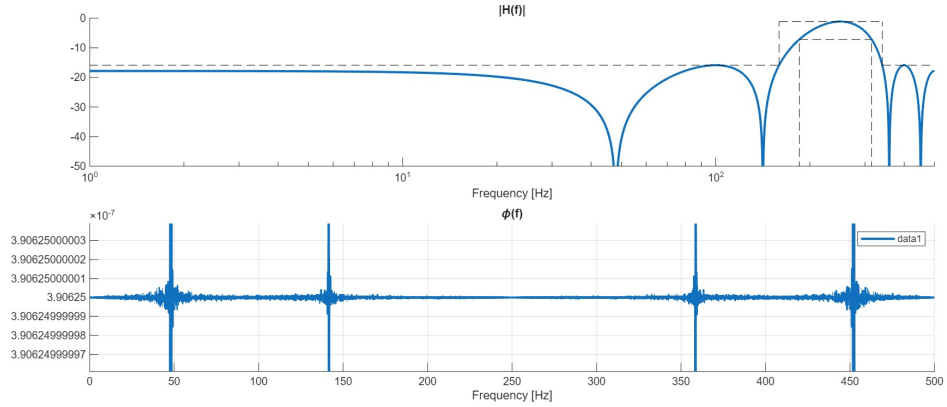

FIGURE 8 – Module de la réponse fréquentielle (haut) et temps de propagation de groupe (bas)

La Figure 8 illustre le comportement du filtre en fréquence. Globalement, les résultats obtenus par simulation sont cohérents avec la théorie, mais avec quelques écarts liés à la troncature.

1. Le Module de la réponse $|H(f)|$

Le graphique du haut montre bien un gabarit de passe-bande centré entre 200 Hz et 300 Hz. Cependant, comme on a utilisé très peu de coefficients ($N = 9$) et une fenêtre rectangulaire simple, le filtre est loin d'être idéal. Les zones de transition sont très larges et on observe des ondulations importantes (phénomène de Gibbs).

Les valeurs relevées par l'algorithme montrent bien ces écarts par rapport aux fréquences de coupure visées :

— Bande basse : $f_{L1} \approx 159{,}5\,\text{Hz}$ et $f_{R1} \approx 185{,}2\,\text{Hz}$ (Pente de montée d'environ $+134{,}6\,\text{dB/dec}$).

— Bande haute : $f_{L2} \approx 314{,}9\,\text{Hz}$ et $f_{R2} \approx 340{,}6\,\text{Hz}$ (Pente de descente d'environ $-256{,}2\,\text{dB/dec}$).

2. Le Temps de propagation de groupe $\tau_g(f)$

Sur le graphique du bas, le temps de propagation de groupe (la dérivée de la phase) est une ligne parfaitement droite. Les infimes variations que l'on voit (de l'ordre de 10^{-11}) viennent uniquement du bruit de calcul du logiciel.

Le temps mesuré est constant et vaut 4 ms. Cela correspond parfaitement au calcul théorique pour ce filtre :

$$\tau_g = \frac{N-1}{2} \cdot T_e = 4 \times 1\,\text{ms} = 4\,\text{ms} \tag{1}$$

Le filtre fonctionne donc comme prévu : étant à phase linéaire, toutes les fréquences du signal subiront le même retard (4 ms), ce qui évite de déformer le signal en sortie.

4 Partie D : Synthèse de filtres à RIF sous Matlab

4.1 Implémentation et choix des paramètres

Dans cette partie, nous avons utilisé Matlab pour concevoir un filtre RIF passe-bande d'ordre $N = 30$. Contrairement à la méthode de la fenêtre vue précédemment, nous avons utilisé ici la fonction `firpm`, qui implémente l'algorithme de Parks-McClellan. Cette méthode permet de concevoir un filtre optimal à ondulations égales.

Afin de respecter le gabarit imposé avec de fortes contraintes d'atténuation dans les bandes coupées, nous avons affecté des poids différents lors de la synthèse : `Wstop = 60` pour les bandes coupées et `Wpass = 1` pour la bande passante. Cela force l'algorithme à privilégier l'atténuation du bruit plutôt que la planéité parfaite de la bande passante.

Le code Matlab correspondant est fourni en Annexe 5.4 (page 31).

4.2 Analyse de la réponse fréquentielle

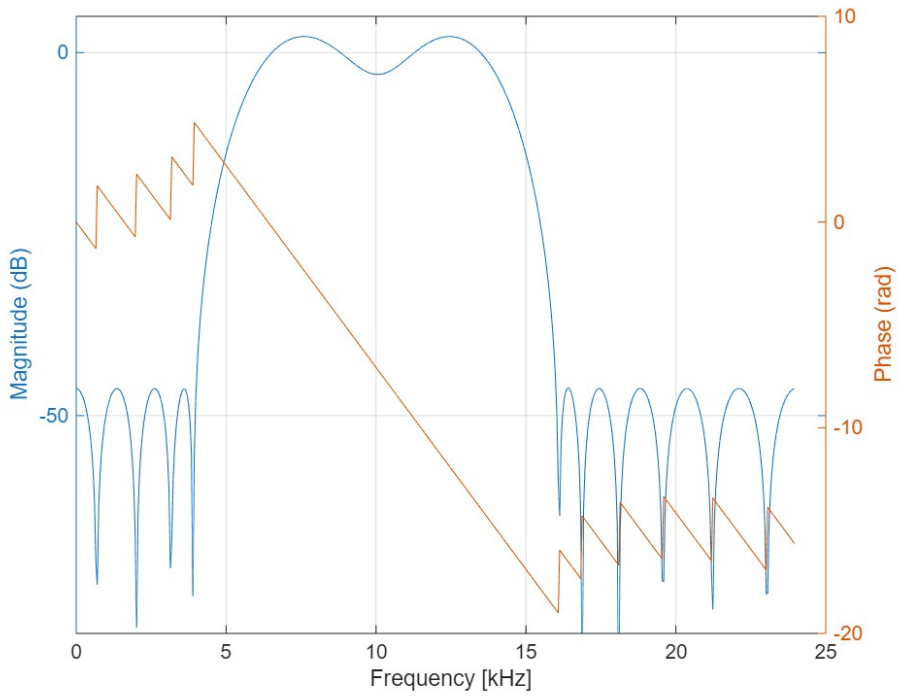

FIGURE 9 – Module (dB) et Phase (rad) de la réponse fréquentielle

La simulation de la réponse fréquentielle (Figure 9) valide notre paramétrage. Le module montre un filtre passe-bande correctement centré entre 6 kHz et 14 kHz. Grâce au poids de 60, l'atténuation dans les bandes coupées est très forte. De plus, on remarque que la courbe de phase (en rouge) est une droite parfaite dans la zone passante. Les

résultats obtenus par simulation sont donc cohérents avec les valeurs théoriques d'un filtre RIF à phase linéaire.

4.3 Réponse impulsionnelle et temps de propagation de groupe

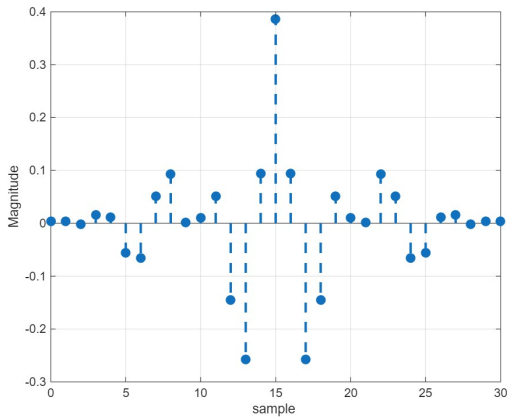

FIGURE 10 – Coefficients du filtre RIF

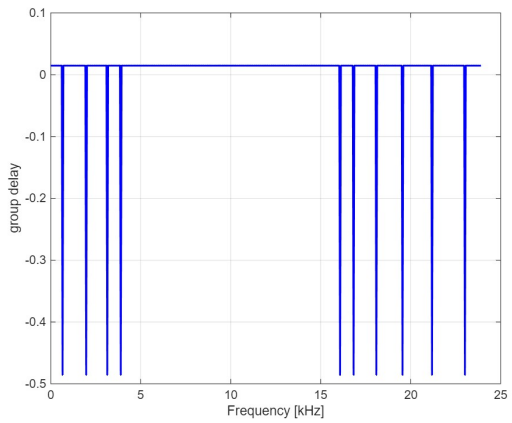

FIGURE 11 – Temps de propagation de groupe

Afin de comprendre d'où vient cette linéarité de phase, nous avons affiché les coefficients du filtre (Figure 10). Pour un ordre $N = 30$, nous avons 31 échantillons. On constate une symétrie parfaite autour de l'échantillon central ($n = 15$).

La Figure 11 vient confirmer ce comportement de manière évidente : le temps de propagation de groupe est strictement constant. La valeur relevée est de 15 échantillons, ce qui correspond exactement au calcul théorique :

$$\tau_g = \frac{N}{2} = \frac{30}{2} = 15 \text{ échantillons} \tag{2}$$

Le montage présente un fonctionnement stable et idéal pour l'audio, car il n'introduit aucune distorsion de phase.

4.4 Validation pratique par filtrage audio

Pour valider le fonctionnement global de notre filtre, nous l'avons appliqué sur un fichier musical échantillonné à 48 kHz. En comparant les spectrogrammes sur la Figure 12, le fonctionnement du passe-bande est très clair. Sur le spectrogramme du bas (signal filtré), toute l'énergie du signal est concentrée exclusivement dans la bande 6 kHz – 14 kHz. Les basses fréquences (sous 4 kHz) et les très hautes fréquences (au-delà de 16 kHz) ont été totalement atténuées par notre filtre. L'écoute auditive vient confirmer la perte des graves et le rendu très "téléphonique" attendu pour ce gabarit.

5 Partie E : Synthèse de filtres à RII sous Matlab

5.1 Choix des paramètres et implémentation

Pour cette dernière partie, nous avons synthétisé un filtre à Réponse Impulsionnelle Infinie (RII) d'ordre 10. L'objectif était d'obtenir une réponse en fréquence similaire à

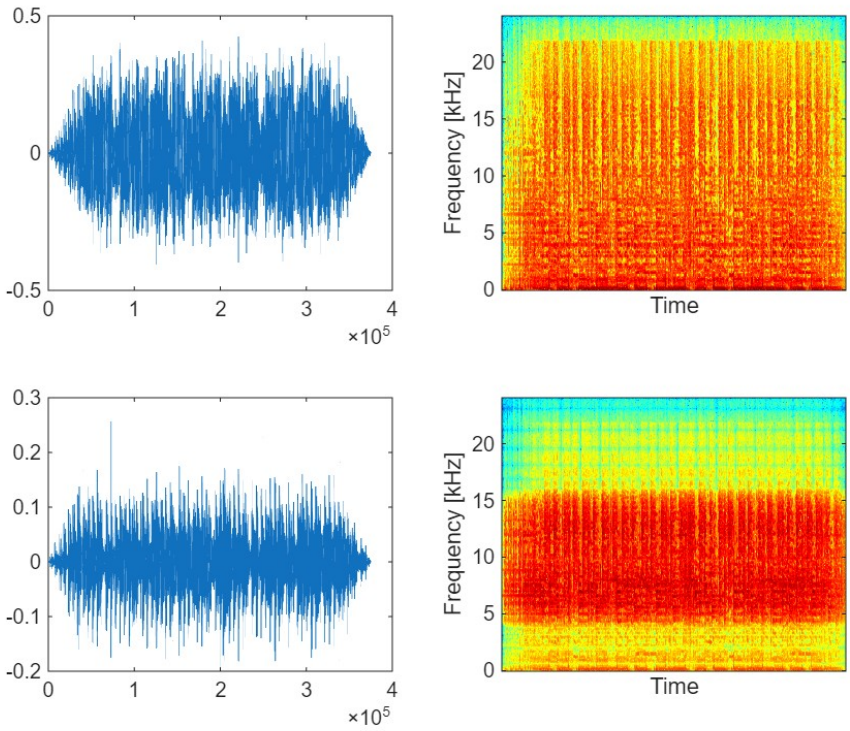

FIGURE 12 – Signal original vs Signal filtré : domaine temporel et spectrogrammes

celle de notre filtre RIF d'ordre 30, afin de comparer les deux technologies.

Nous avons utilisé l'architecture de *Chebyshev de type 2*, qui présente l'avantage de n'avoir aucune ondulation dans la bande passante (gabarit plat) tout en garantissant une atténuation forte (réglée ici sur $A_{stop} = 48\,\text{dB}$) dans les bandes coupées. Pour assurer la stabilité numérique lors de l'application du filtre, nous avons converti la fonction de transfert sous forme de cellules du second ordre (SOS - *Second Order Sections*).

Le code Matlab correspondant est fourni en Annexe 5.4 (page 32).

Démarche de choix des paramètres

Pour ce filtre de Chebyshev de type II, les paramètres ont été fixés selon une démarche visant à surpasser les performances du filtre RIF de la partie précédente, tout en optimisant les ressources utilisées.

— **Le choix de l'ordre $N = 10$:** En technologie RII, l'ordre est directement lié au nombre de pôles du système. Un ordre de 10 est ici suffisant pour obtenir une pente de coupure bien plus raide qu'un filtre RIF d'ordre 30. L'objectif est de trouver l'ordre minimal garantissant une transition nette entre bande passante et bande coupée, sans risquer d'instabilité numérique liée à une structure trop complexe.

— **La définition des bandes de transition :** Les fréquences d'arrêt ont été fixées à $f_{\text{stop1}} = 4200\,\text{Hz}$ et $f_{\text{stop2}} = 16600\,\text{Hz}$. En laissant une zone de transition d'environ 2 kHz de part et d'autre de la bande passante utile (6 kHz – 14 kHz), on facilite le travail de l'algorithme de synthèse tout en maintenant un gabarit proche de celui

du filtre RIF conçu précédemment.

— **L'atténuation** $A_{\text{stop}} = 48\,\text{dB}$ **:** Ce paramètre assure que les fréquences rejetées voient leur amplitude divisée par un facteur d'environ 250. Pour un signal audio, ce niveau d'atténuation permet de rendre les composantes indésirables négligeables à l'oreille, tout en restant réalisable avec un ordre faible.

Dans la pratique, ce dimensionnement peut s'effectuer par une approche itérative : on ajuste l'ordre et l'atténuation jusqu'à obtenir un spectre fréquentiel visuellement conforme au gabarit attendu, en s'assurant à chaque étape que les pôles restent à l'intérieur du cercle unité.

5.2 Analyse de la réponse temporelle et fréquentielle

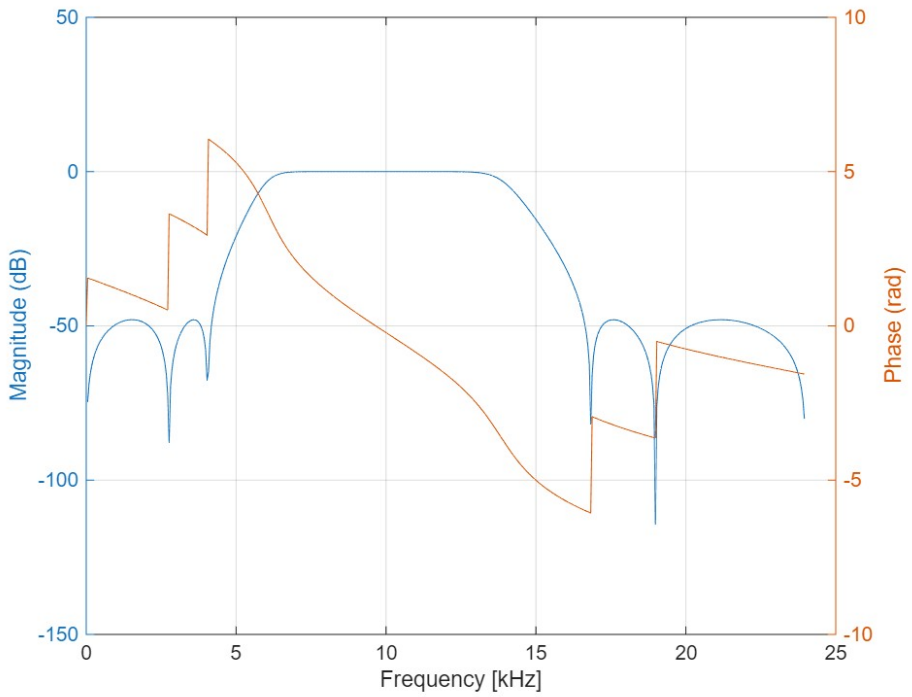

FIGURE 13 – Réponse impulsionnelle

L'analyse de ces trois graphiques met en évidence les différences majeures entre un filtre RII et un filtre RIF :

— **Dans le domaine temporel (Figure 13)** : La réponse impulsionnelle n'est pas symétrique et s'amortit très lentement vers zéro. C'est le comportement typique d'un système bouclé (feedback).

— **Dans le domaine fréquentiel (Figures 14 et 15)** : Le module (en bleu) montre un filtrage très efficace malgré un ordre faible ($N = 10$). En revanche, la phase (en rouge) n'est absolument pas linéaire. Cela se traduit par un temps de propagation de groupe fortement instable, avec de grands pics atteignant 15 ms aux abords des fréquences de coupure. Contrairement au RIF, ce filtre va donc introduire de la distorsion de phase.

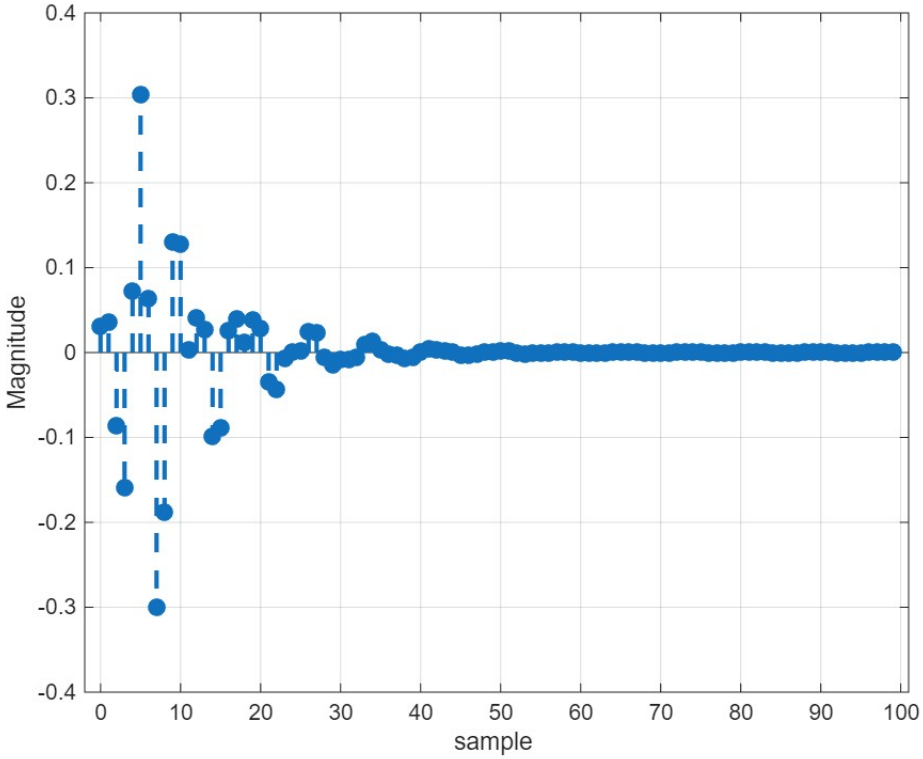

FIGURE 14 – Module et phase

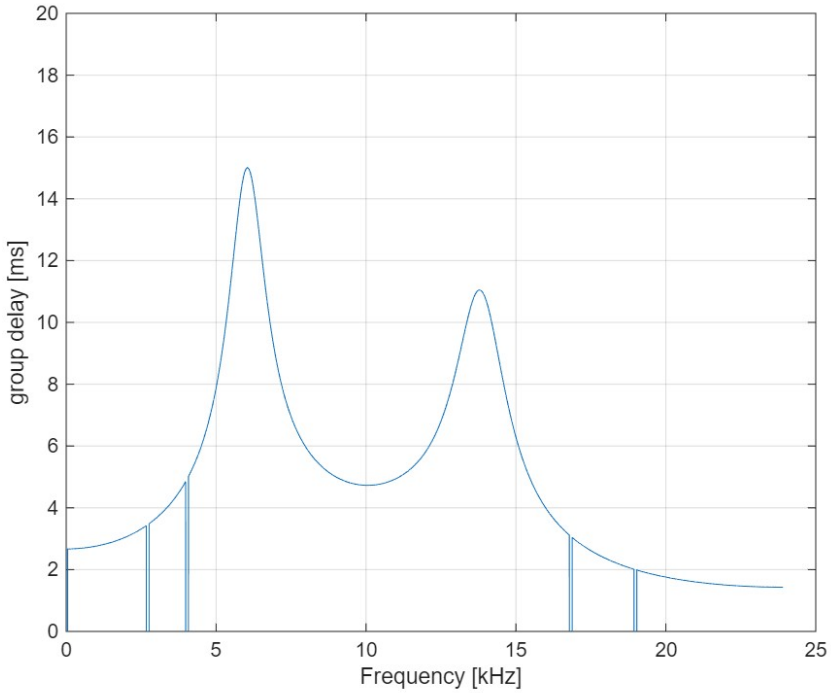

FIGURE 15 – Temps de propagation

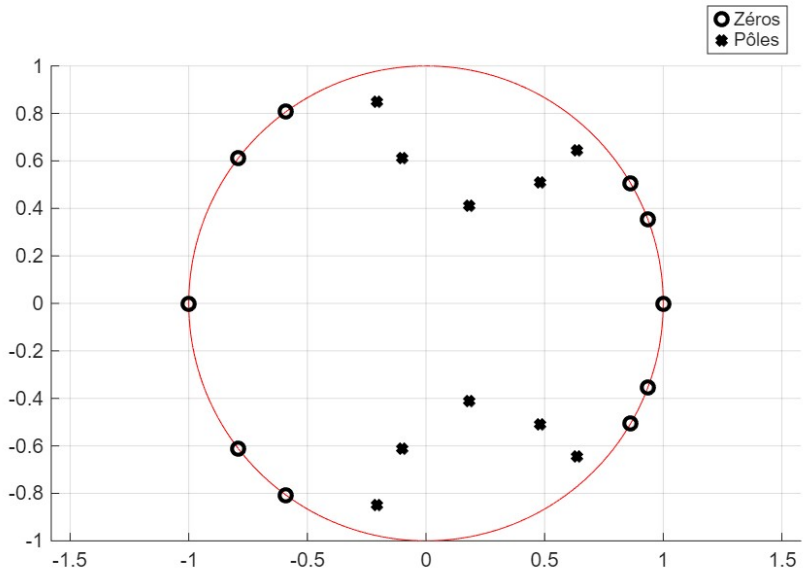

FIGURE 16 – Diagramme des pôles et zéros du filtre RII sur le plan complexe

5.3 Étude de la stabilité : Plan complexe

Puisqu'un filtre RII possède une boucle de retour, il présente un risque d'instabilité. La théorie stipule qu'un système discret est stable si et seulement si tous les pôles de sa fonction de transfert se situent strictement à l'intérieur du cercle unité.

La simulation (Figure 16) valide ce principe théorique : toutes les croix noires (les pôles) sont bien contenues à l'intérieur du cercle rouge de rayon 1. Le montage simulé est donc parfaitement stable. Les zéros (cercles noirs) situés exactement sur le cercle unité sont responsables des "creux" d'atténuation infinis que l'on observe dans la bande coupée du filtre.

5.4 Validation pratique par filtrage audio

Nous avons appliqué ce filtre sur le même fichier musical que précédemment en utilisant la fonction `sosfilt`. Sur les spectrogrammes de la Figure 17, les résultats obtenus sont visuellement identiques à ceux du filtre RIF : la bande utile est très bien isolée.

En conclusion, la simulation montre que l'on peut obtenir un filtrage tout aussi brutal et efficace qu'un RIF d'ordre 30 avec un RII d'ordre 10 seulement. Le RII est donc beaucoup moins coûteux en ressources de calcul, ce qui est un avantage majeur en temps réel. Le seul compromis à accepter est la déformation de phase générée aux alentours des fréquences de coupure.

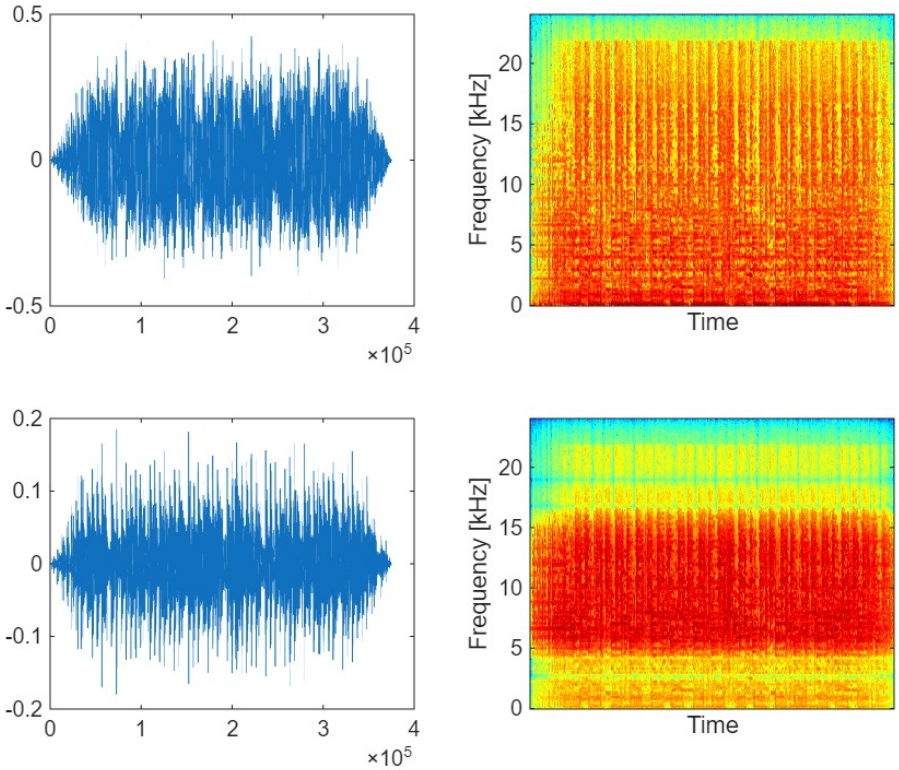

FIGURE 17 – Signal original vs Signal filtré par RII : domaine temporel et spectrogrammes

Conclusion

Ce travail dirigé nous a permis de parcourir de manière progressive et cohérente l'ensemble de la chaîne de synthèse d'un filtre numérique, depuis les bases théoriques jusqu'à la validation expérimentale sur signal réel.

Sur la méthode de la fenêtre (Partie A)

La première partie a mis en évidence le compromis fondamental inhérent à tout filtre RIF synthétisé par fenêtrage. La troncature de la réponse impulsionnelle idéale est inévitable, et le phénomène de Gibbs qu'elle induit ne peut être éliminé, seulement atténué. Parmi les 14 fenêtres étudiées, aucune n'est universellement optimale : le choix dépend toujours de l'application visée, selon que l'on cherche à maximiser la sélectivité (fenêtre rectangulaire), à mesurer une amplitude précise (Flat-top), ou à rejeter fortement le bruit hors bande (Blackman-Harris, Blackman-Nuttall).

Sur la phase linéaire (Partie B)

La démonstration rigoureuse a établi qu'une réponse impulsionnelle symétrique autour de son coefficient central est la condition nécessaire et suffisante pour obtenir une phase linéaire. Cette propriété se traduit par un temps de propagation de groupe constant $\tau_g = (N-1)/2 \cdot T_e$, garantissant l'absence de distorsion temporelle pour toutes les composantes fréquentielles du signal.

Sur la synthèse pratique (Parties C, D et E)

L'implémentation sous Matlab des trois filtres synthétisés a permis de confronter la théorie à la simulation. Les résultats obtenus sont cohérents avec les prédictions analytiques dans chaque cas.

La comparaison entre le filtre RIF d'ordre 30 (Partie D) et le filtre RII d'ordre 10 (Partie E) illustre de manière concrète le compromis central entre ces deux architectures. Le filtre RIF, grâce à sa symétrie, offre une phase parfaitement linéaire et une stabilité garantie, au prix d'un ordre élevé et donc d'un coût calculatoire plus important. Le filtre RII, quant à lui, atteint des performances de filtrage comparables avec un ordre bien inférieur, mais au détriment de la linéarité de phase : le temps de propagation de groupe présente des pics importants aux abords des fréquences de coupure, ce qui introduit une distorsion temporelle non négligeable dans un contexte audio ou de communication.

Bilan

En définitive, la maîtrise de ces outils de synthèse ne se réduit pas à l'utilisation de fonctions Matlab. Elle suppose une compréhension approfondie des phénomènes sous-jacents — Gibbs, causalité, stabilité, phase linéaire — qui conditionnent le choix de l'architecture et des paramètres adaptés à chaque application. Les notions abordées dans ce TDAO constituent des fondements essentiels pour la conception de systèmes de communication numérique performants.

Annexes — Codes Matlab

Code MATLAB de la partie A : question 7 :

```
clear; close all;

% Param tres du filtre
Fc = 250;  % Fr quence de coupure en Hz
Fe = 2000; % Fr quence d'   chantillonnage    en Hz

% Diff rentes longueurs de troncature    tester
N_values = [16, 51, 101];

%% 1. Analyse temporelle : R ponse impulsionnelle
figure(1); clf; hold on;
for i = 1:length(N_values)
    N = N_values(i);
    % Cr ation d'un axe temporel sym trique centr  sur 0
    if mod(N,2) == 0
        n = -N/2 : N/2-1;
    else
        n = -(N-1)/2 : (N-1)/2;
    end

    % Calcul du sinus cardinal
    h = 2*Fc/Fe * sinc(2*n*Fc/Fe);
    stem(n, h, 'DisplayName', ['N = ' num2str(N)]);
end
grid on; hold off;
title('R ponse impulsionnelle tronqu e d''un filtre passe-bas');
xlabel('Num ro d''  chantillon    (n)');
ylabel('Amplitude');
legend;

%% 2. Analyse fr quentielle : Spectre et D lai de groupe
figure(2); clf;
for i = 1:length(N_values)
    N = N_values(i);
    NB = N * 20; % Zero-padding pour affiner la FFT

    % Recalcul de h(n) pour la FFT
    if mod(N,2) == 0, n = -N/2 : N/2-1; else, n = -(N-1)/2 : (N-1)/2; end
    h = 2*Fc/Fe * sinc(2*n*Fc/Fe);

    H = fft(h, NB);
    f = linspace(0, 1, NB) * Fe; % Axe des fr quences
```

```

    % Affichage de l'Amplitude
    subplot(2, length(N_values), i); hold on;
    plot(f, abs(H), 'linewidth', 2, 'DisplayName', ['N = ' num2str(N)]);
    xlabel('Fr quence (Hz)'); ylabel('Amplitude');
    xlim([0, Fe/2]); grid on; legend;

    % Affichage du D lai de groupe
    subplot(2, length(N_values), i + length(N_values)); hold on;
    delay = -diff(unwrap(angle(H))) / (2*pi*Fe);
    plot(f(2:end), delay, 'linewidth', 2, 'DisplayName', ['N = ' num2str(N)]);
    xlabel('Fr quence (Hz)'); ylabel('D lai de groupe (s)');
    xlim([0, Fe/2]); ylim([-3 3]*1e-4); grid on; legend;
end
```

Code Matlab - Troncature et phénomène de Gibbs

Code MATLAB de la partie A : question 8 :

```
clc;
clear;
close all;
%pkg load signal; % Pour GNU Octave

% Fen tres d'apodisation :
% rectangulaire, triangulaire, Hann, Hamming, Blackman,
% Gaussienne, Welch, puissance de sinus, Flap-top, Tukey,
% Lanczos, Kaiser, BlackmanHarris,
% BlackmanNuttall

%% D finition des fen tres d'apodisation manquantes

function w = welchwin(n)
  m = floor(n/2); k = (0:m-1)';
  w = 1 - (2*k/n - 1).^2;
  if(mod(n, 2) ~= 0), w = [w; 1]; end
  w = [w; flipud(w(1:m, 1))];
end

function w = lanczoswin(n)
  w = sinc(2*[0:n-1]/(n-1)-1);
  w=w';
end

function w = sinepow(n,alpha)
  w = sin(pi*[0:n-1]/(n-1)).^alpha;
```

```
  w = w';
end

%% Fonctions permettant l'analyse des performances de fen tres

function first_minimum_index = get_first_minimum_index (bins)
        prev_bin = bins(1);
        for i = 1:length(bins)
                current_bin = bins(i);

                if (current_bin > (prev_bin + 0.01))     % To prevent bug where 10.00 > 10.00 is considered true
                        first_minimum_index = i;
                        break;
                else
                        prev_bin = current_bin;
        end
    end
end

function width_bin = get_first_lobe_width (bins)
        width_bin = 2 * get_first_minimum_index(bins);
end

function att_dB = get_secundary_lobe_attenuation (bins)
        first_minimum_index = get_first_minimum_index(bins);

        % Keep the data for the second to fourth lobe
        filtered_bins = bins(first_minimum_index:first_minimum_index*4);

        att_dB = max(filtered_bins);
end

function slope = get_Gibbs_slope (bins)
        [peak_values, peak_indexes] = findpeaks(bins, 'NPeaks', 10);

    slope = (peak_values(7) - peak_values(6)) / (log2(peak_indexes(7)) - log2(peak_indexes(6)));
end


%% Calcul, trac des graphiques, et valuation des performances pour chaque fen tre

% Param tres
```

```
N=101;
NB=10240;

% Calcul des s quences     chantillonnes     des fen tres d'
    apodisation
ff=[rectwin(N),triang(N),hann(N),hamming(N)];
ff=[ff,blackman(N),gausswin(N),welchwin(N),sinepow(N,4)];
ff=[ff,flattopwin(N),tukeywin(N),lanczoswin(N),kaiser(N)];
ff=[ff,blackmanharris(N),nuttallwin(N)];

winNames={'rectangulaire','triangulaire','Hann','Hamming','
    Blackman','Gaussienne','Welch','puissance de sinus','flat-top
    ','Tukey','Lanczos','Kaiser','Blackman-Harris','Blackman-
    Nuttall'};

% 7 premi res fen tres
figure(1);clf;hold on;
for it=1:7
    w=ff(:,it);
    [f,W]= myResp(w,NB);

    % Trac  de la r ponse impulsionnelle
    subplot(2,7,it);hold on;
        plot([0:N-1],w);
    title({'R ponse impulsionnelle de ',['"',winNames{it},'"'
        ]});

    % Trac  de la fonction de transfert
        bins = 20*log10(abs(W));
    subplot(2,7,7+it);hold on;
    plot(f, bins);
    title({'Fonction de transfert de ',['"',winNames{it},'"']});
    xlim([0,0.1]);

    % Affichage des performances de la fen tre
    disp([newline winNames{it}, ' :']);

        disp(['First lobe width : ', num2str(
            get_first_lobe_width(bins) / N), ' bins / N']);
        disp(['Secundary lobe attenuation : ', num2str(
            get_secundary_lobe_attenuation(bins) - bins(1)), 'dB'
            ]);
    disp(['Gibbs zone slope : ', num2str(get_Gibbs_slope(bins)),
        'dB/oct']);

end

% 7 derni res fen tres
```

```
figure(2);clf;hold on;
for it=1:7
    w=ff(:,it+7);
    [f,W]= myResp(w,NB);

    % Trac  de la r ponse impulsionnelle
    subplot(2,7,it);hold on;
        plot([0:N-1],w);
    title({'R ponse impulsionnelle de ',['"',winNames{it+7},'"']});

    % Trac  de la fonction de transfert
        bins = 20*log10(abs(W));
    subplot(2,7,7+it);hold on;
    plot(f, bins);
    title({'Fonction de transfert de ',['"',winNames{it+7},'"']});
    xlim([0,0.1]);

    % Affichage des performances de la fen tre
    disp([newline winNames{it+7}, ' :']);

        disp(['First lobe width : ', num2str(get_first_lobe_width(bins) / N), ' bins / N']);
        disp(['Secundary lobe attenuation : ', num2str(get_secundary_lobe_attenuation(bins) - bins(1)), ' dB']);
    disp(['Gibbs zone slope : ', num2str(get_Gibbs_slope(bins)), 'dB/oct']);
end
```

Code Matlab - Analyse des fenêtres d'apodisation

Code MATLAB de la partie A : question 10 :

```
clc; clear; close all;

% -- D finition des param tres du signal --
Fe = 1000;            % Fr quence d'  chantillonnage    (Hz)
duree_signal = 1;     % Dur e d'observation (sec)
t = 0:1/Fe:(duree_signal - 1/Fe);
N = length(t);

% -- Cr ation du signal polychromatique --
A1 = 1;   f1 = 12.5;  % Fr quence "non-enti re" sur 1s
A2 = 0.6; f2 = 50;    % Fr quence "enti re" sur 1s
A3 = 0.4; f3 = 110.1; % Fr quence "non-enti re" sur 1s

```

```
signal = A1*sin(2*pi*f1*t) + A2*sin(2*pi*f2*t) + A3*sin(2*pi*f3*t);

% La Figure 1 trace le signal temporel brut et sa FFT classique ...

% —— Application des fen tres d'apodisation ——
windows = [rectwin(N), flattopwin(N), tukeywin(N), blackmanharris(N)];
windows_name = {'rectangulaire', 'Flat-top', 'Tukey', 'Blackman-Harris'};

figure(2);
for i = 1:4
    % Multiplication temporelle du signal par la fen tre
    signal_apodise = signal .* windows(:, i)';

    % Trac  temporel
    subplot(2, 4, i);
    plot(t, signal_apodise);
    title(['Fen tre ' windows_name{i}]); xlabel('temps (s)');

    % Trac  fr quentiel (Spectre normalis )
    subplot(2, 4, i + 4);
    fft_norm = 20*log10(abs(fft(signal_apodise)) .* 2 ./ sum(windows(:, i)));
    plot(f, fft_norm);
    title('Spectre'); xlabel('Fr quence (Hz)'); ylabel('Amplitude (dB)');
    xlim([0 120]); ylim([-150 0]);
end
```

Code Matlab - Apodisation d'un signal transitoire

Code MATLAB de la partie C :

```
clc; clear;

% Param tres du filtre
Fe = 1000; % Fr quence d'  chantillonnage  en Hz
Te = 1/Fe;
fc1 = 200; % Fr quence de coupure basse
fc2 = 300; % Fr quence de coupure haute
N = 9;     % Nombre de coefficients
NB = 10240; % Nombre de points pour la FFT

% —— G n ration de la r ponse impulsionnelle causale ——
n = 0:N-1;
```

```
m = n - (N-1)/2; % Centrage pour la formule du filtre
h = (sin(2*pi*fc2*m*Te) - sin(2*pi*fc1*m*Te)) ./ (pi * m);
h(m == 0) = 2*(fc2 - fc1)*Te; % Gestion de la limite en 0

% Figure 1 : Affichage de la r ponse impulsionnelle
figure(1); clf; hold on;
stem(n, h, '--o', "filled", "LineWidth", 2);
grid on; hold off;
xlabel("n (   chantillons   )");
ylabel("Amplitude \hat{h}(n)");
title("R ponse impulsionnelle causale du filtre");

% -- Passage dans le domaine fr quentiel --
H = fft(h, NB);

% La suite du script trace le module et le temps de propagation
    de groupe...
```

Code Matlab - Synthèse et analyse du filtre RIF

Code MATLAB de la partie D :

```
clear; clc; close all;
NB = 512;
Fs = 48000;
N      = 30;
Fstop1 = 4000;
Fpass1 = 6000;
Fpass2 = 14000;
Fstop2 = 16000;
Wstop1 = 60;
Wpass  = 1;
Wstop2 = 60;
dens   = 20;

% Synth se avec firpm : N = ordre, [freqs normalis es], [gains
    ], [poids]
b  = firpm(N, [0 Fstop1 Fpass1 Fpass2 Fstop2 Fs/2]/(Fs/2), ...
            [0 0 1 1 0 0], [Wstop1 Wpass Wstop2], {dens});
Hd = dfilt.dffir(b); % Cr ation de l'objet filtre

% Calcul de la r ponse fr quentielle
[H,w] = freqz(b,1,NB);
freq = w./(2 * pi) .* Fs ./ 1000;
```

Code Matlab - Synthèse du filtre RIF optimal

Code MATLAB de la partie E :

```
clear; clc;
NB = 512;
Fs = 48000;
N      = 10;      % Ordre du filtre
Fstop1 = 4200;    % Coupure basse
Fstop2 = 16600;   % Coupure haute
Astop  = 48;      % Attenuation en bande coup e (dB)

% Synth se du filtre Chebyshev type 2
h  = fdesign.bandpass('N,Fst1,Fst2,Ast', N, Fstop1, Fstop2,
    Astop, Fs);
Hd = design(h, 'cheby2');

% Calculs pour l'affichage (R ponse impulsionnelle et
    fr quentielle)
[hn,idx] = impz(Hd);
[H,w_Hz] = freqz(Hd,[],Fs);
freq = w_Hz / 1000;

% Conversion en forme canonique SOS (Second-Order Sections) pour
    la stabilit
[num, den] = tf(Hd);
sos = tf2sos(num, den);
[z, p, k] = sos2zp(sos, 1); % Extraction des p les et z ros
```

Code Matlab - Synthèse et analyse du filtre RII